LAB 02 REPORT TEMPLATE

Advanced API Testing & Automation with Postman (Medusa.js Staging)

Course Name & Code:	Advanced Testing & QA Automation
Team Name / ID:	Team01
Jira / Trello Board Link:	https://vanlangsaigon-team1-medusa.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog
GitHub Repository URL:	https://github.com/2500IT10003/ATQAA_Team01_Medusa_Automation

SECTION I: TEAM ROSTER & PEER ASSESSMENT MATRIX

Complete the allocation table below. All team members must agree on contribution percentages and sign prior to submission.

No.	Student Name	Student ID	Assigned Responsibilities	Contrib.	Sign
1	Khin Thu Zar Aung	2500IT10003	Lead QE & Playwright Specialist** | Web UI Page Object Model (POM), E2E Checkout flows, Admin Fulfillment verification, CI/CD Pipeline orchestration	33.3%	Zar
2	Yu Yu Khine	2500IT10010	API Automation Engineer** | Postman Collections, Dynamic JS chaining, Pre-request & Test scripts, Newman CLI runner integration	33.3%	Khine
3	Kaung Satt Win	2500IT10002	Synthetic Data & Performance Specialist** | GDPR Synthetic Test Data generation (1,000+ rows), k6 Load Engine scripts, Stress/Soak test execution	33.3%	Kaung

SECTION II: WORKSPACE & ENVIRONMENT CONFIGURATION

Configure Postman Environment variables to ensure zero hardcoded values in API requests.

Variable	Initial Value	Current Value	Variab

Name			leScope& Description
baseUrl	https://atqaa-medusa-store.onrender.com/store	https://atqaa-medusa-store.onrender.com/store	Global/Env:StorefrontRESTAP

			IbaseURL
adminBaseUrl	https://atqaa-medusa-store.onrender.com	https://atqaa-medusa-store.onrender.com	Global/Env: AdminAPIbaseURL
variantId	variant_01M4CX92CRHTENEF0SNNWNR6EF	variant_01M4CX92CRHTENEF0SNNWNR6EF	Env: DynamicPr

			oductVariantIDforcartlineitems
cartId	cart_01M5HYCDMDM3SEV50Y6V89V5YQ	cart_01M5HYCDMDM3SEV50Y6V89V5YQ	Env: DynamicProductCartID

			maintainedacrossthecheckoutsession
apiToken	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY3Rvcl9pZCI6InVzZXJfMDFNNEJEU0s0N0c4WTU2OThYQ1NUQ1Y2WDIiLCJhY3Rvcl90eXBlIjoidXNlciIsImF1dGhfaWRlbnRpdHlfaWQiOiJhdXRoaWRfMDFNNEJEU0s5MDFNSFA1WFNYTlI0UTAzQTEiLCJhdXRoX3Byb3ZpZGVyIjoiZW1haWwxYXNzIiwiYXBwX21ldGFkYXRhIjp7InVzZXJfaWQiOiJ1c2VyXzAxTTRCRFNLNDdHOFk1Njk4WENTVENWNlgyIn0sInVzZXJfbWV0YWRhdGEiOnt9LCJpYXQiOjE3OTI2ODAxMDgsImV4cCI6MTc5Mjc2NjUwOH0.qz9ZlKv7tbAT1YUhwAZ00igypWF4nmwxds-PfbhI5VE	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY3Rvcl9pZCI6InVzZXJfMDFNNEJEU0s0N0c4WTU2OThYQ1NUQ1Y2WDIiLCJhY3Rvcl90eXBlIjoidXNlciIsImF1dGhfaWRlbnRpdHlfaWQiOiJhdXRoaWRfMDFNNEJEU0s5MDFNSFA1WFNYTlI0UTAzQTEiLCJhdXRoX3Byb3ZpZGVyIjoiZW1haWwxYXNzIiwiYXBwX21ldGFkYXRhIjp7InVzZXJfaWQiOiJ1c2VyXzAxTTRCRFNLNDdHOFk1Njk4WENTVENWNlgyIn0sInVzZXJfbWV0YWRhdGEiOnt9LCJpYXQiOjE3OTI2ODAxMDgsImV4cCI6MTc5Mjc2NjUwOH0.qz9ZlKv7tbAT1YUhwAZ00igypWF4nmwxds-PfbhI5VE	Env:JWTBearerTo

			kenforAdminauthorization

SECTION III: DYNAMIC API CHAINING WORKFLOW (5 REQUESTS)

Detail the sequence of 5 E2E chained API requests simulating a complete user purchasing journey.

Seq	HTTP Method & Endpoint	Input Variables	Extracted Dynamic Output	Verification Status
1	POST {{adminBaseUrl}}/auth/user/emailpass	Headers: Content-Type: application/json Body(raw-JSON): adminEmail, adminPassword	apiToken	200 OK
2	GET {{baseUrl}}/products	Headers:	variantId	200 OK

		Content-Type: application/json		
3	POST {{baseUrl}}/carts	Headers: Content-Type: application/json Body(raw-JSON): {}	cartId	200 OK
4	POST {{baseUrl}}/carts/{{cartId}}/line-items	Headers: Content-Type: application/json Body(raw-JSON) { "variant_id": "{{variantId}}", "quantity": 1 }	line-items	200 OK
5	GET {{baseUrl}}/carts/{{cartId}}	Headers: Content-Type: application/json	Fetch Updated Cart,Validated	200 OK

SECTION IV: MULTI-TIER JAVASCRIPT ASSERTIONS & CODE SNIPPETS

Paste key JavaScript assertion snippets implemented in Postman Tests tab for all 5 requests.

Seq 1:

```
pm.test("Status code is 200 OK", function(){
pm.response.to.have.status(200)
});
pm.test('SLA < 2000ms', function () {
pm.expect(pm.response.responseTime).to.be.below(2000);
});
var jsonData = pm.response.json();
pm.environment.set("apiToken", jsonData.token || pm.response.headers.get("Set-Cookie"));
```

```
pm.test("Tier 2: token is a non-null string with JWT format", function () {
const token = pm.environment.get("apiToken");
pm.expect(token).to.be.a("string").and.not.empty;
pm.expect(token.split(".")).to.have.length(3);
});
// ===== TIER 3 =====
const authSchema = { type: "object", required: ["token"],
properties: { token: { type: "string", minLength: 20 } } };
pm.test('Schema Valid', function()
{pm.response.to.have.jsonSchema(authSchema); });
```

Seq 2:

```
pm.test("Status code is 200 OK", function(){
pm.response.to.have.status(200)
});
pm.test('SLA < 2000ms', function () {
pm.expect(pm.response.responseTime).to.be.below(2000);
});
const productsSchema = {
type: "object", required: ["products"],
properties: { products: { type: "array", minItems: 1, items: {
type: "object", required: ["id", "title", "variants"],
properties: { id: { type: "string" }, title: { type: "string" },
variants: { type: "array", minItems: 1, items: { type: "object", required: ["id"],
properties: { id: { type: "string" } } } } } }
} } }
};
pm.test('Schema Valid', function()
{pm.response.to.have.jsonSchema(productsSchema); });
```

Seq 3:

```
pm.test("Status code is 200 or 201", function(){
pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});
pm.test('SLA < 2000ms', function () {
pm.expect(pm.response.responseTime).to.be.below(2000);
});
var jsonData = pm.response.json();
pm.environment.set("cartId", jsonData.cart.id)
const cartSchema = {
type: "object", required: ["cart"],
properties: { cart: { type: "object",
required: ["id", "region_id", "currency_code", "items", "total"],
properties: {
id: { type: "string", pattern: "^cart_" },
region_id: { type: "string" }, currency_code: { type: "string" },
total: { type: "number" },
items: { type: "array", items: { type: "object",
required: ["id", "variant_id", "quantity", "unit_price"],
properties: { id: { type: "string" }, variant_id: { type: "string" },
quantity: { type: "integer", minimum: 1 }, unit_price: { type: "number" } } } }
} } }
};
pm.test('Schema Valid', function()
```

```
{pm.response.to.have.jsonSchema(cartSchema); });
```

Seq 4:

```
// Tier 1: Verify HTTP Status Code (Return 200 OK or 201 Created)
pm.test("Tier 1: Status code is 200 or 201", function () {
pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

// Tier 2: Verify SLA Performance (< 2000ms)
pm.test("Tier 2: Response time is below 2000ms SLA", function () {
pm.expect(pm.response.responseTime).to.be.below(2000);
});

// Tier 3: Verify Content-Type Header
pm.test("Tier 3: Content-Type is application/json", function () {
pm.response.to.have.header("Content-Type");
});

// Tier 4: Verify Logic - Item Line is not empty
pm.test("Tier 4: Line item array is updated and non-empty", function () {
var jsonData = pm.response.json();
pm.expect(jsonData.cart.items).to.be.an("array").that.is.not.empty;
});
```

Seq 5:

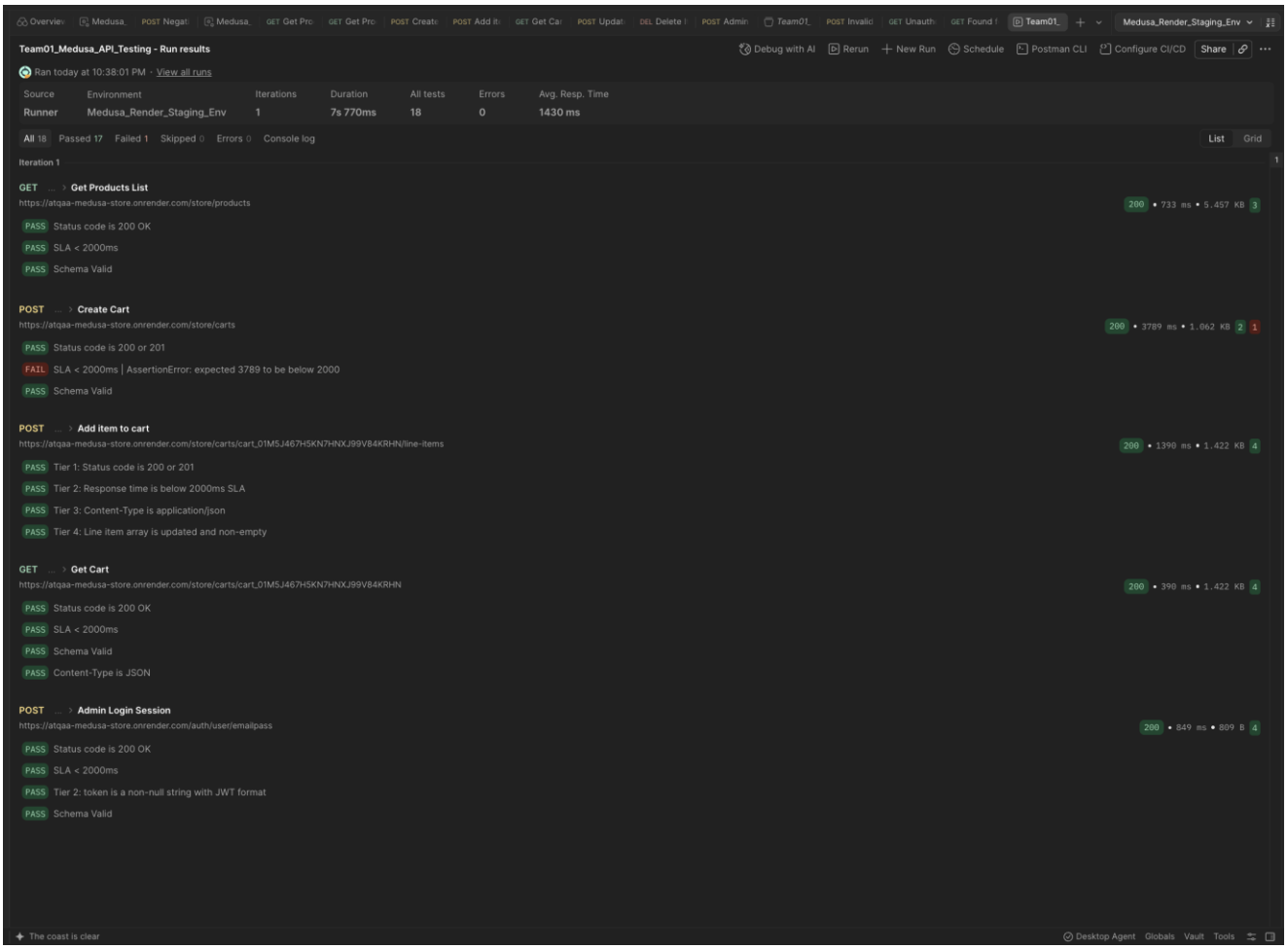
```
pm.test("Status code is 200 OK", function () {
pm.expect(pm.response.code).to.equal(200);
});
pm.test('SLA < 2000ms', function () {
pm.expect(pm.response.responseTime).to.be.below(2000);
});
var jsonData = pm.response.json();
pm.environment.set("cartId", jsonData.cart.id)
const cartSchema = {
type: "object", required: ["cart"],
properties: { cart: { type: "object",
required: ["id", "region_id", "currency_code", "items", "total"],
properties: {
id: { type: "string", pattern: "^cart_" },
region_id: { type: "string" }, currency_code: { type: "string" },
total: { type: "number" },
items: { type: "array", items: { type: "object",
required: ["id", "variant_id", "quantity", "unit_price"],
properties: { id: { type: "string" }, variant_id: { type: "string" },
quantity: { type: "integer", minimum: 1 }, unit_price: { type: "number" } } } } }
} } }
};
pm.test('Schema Valid', function()
{pm.response.to.have.jsonSchema(cartSchema); });

pm.test("Content-Type is JSON", function () {
pm.response.to.have.header("Content-Type");
pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});
```

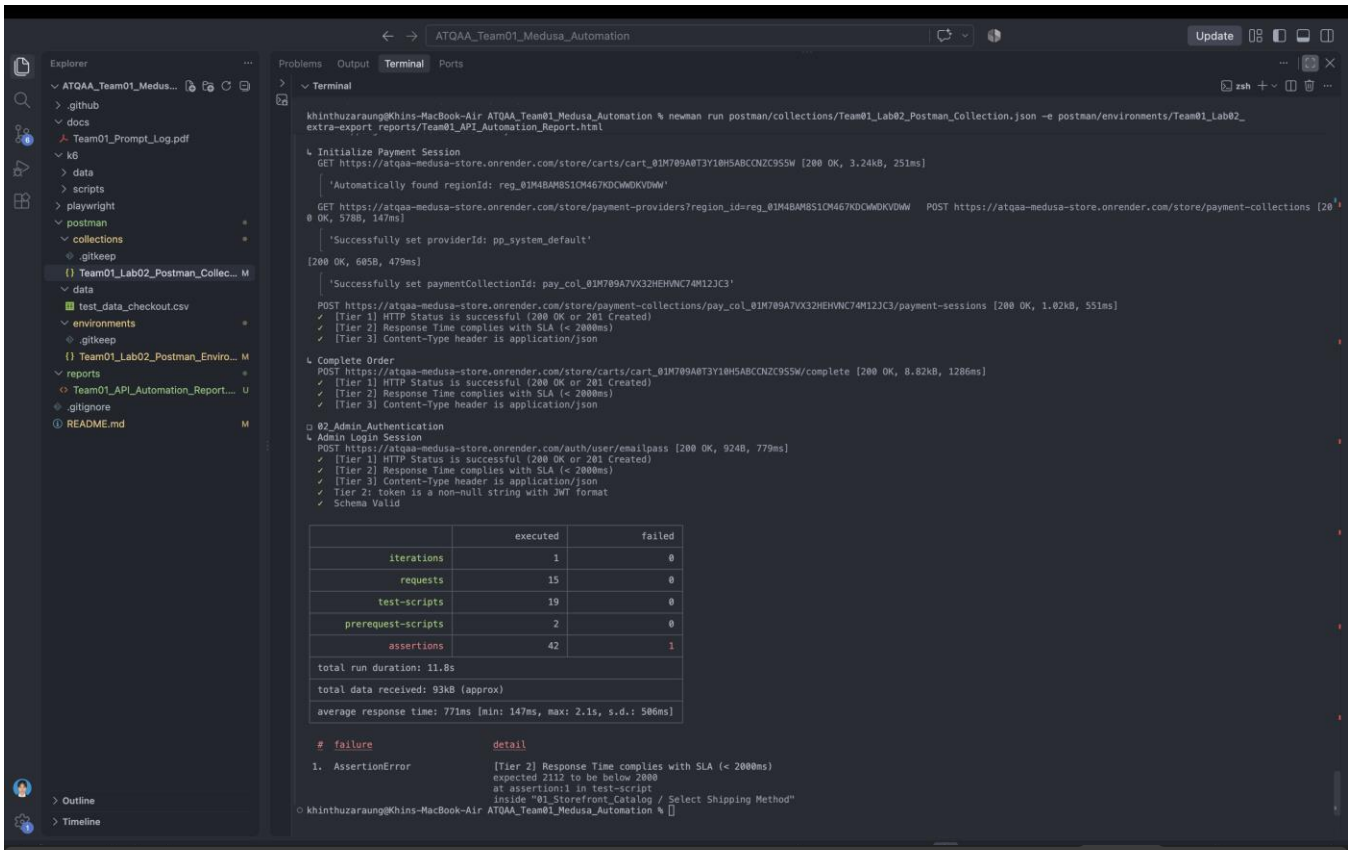
SECTION V: COLLECTION RUNNER & NEWMAN CLI EXECUTION EVIDENCE

Attach evidence screenshots of Postman Collection Runner passing 100% and Newman CLI terminal output.

[INSERT SCREENSHOT 1: Postman Collection Runner — 100% PASSED Results]



[INSERT SCREENSHOT 2: Newman CLI Execution in Terminal / HTML Report Output]


ATQAA_Team01_Medusa_Automation
Update
Explorer
Problems
Output
Terminal
Ports
Terminal
khinthuzaraung@Khins-MacBook-Air ATQAA_Team01_Medusa_Automation % newman run postman/collections/Team01_Lab02_Postman_Collection.json -e postman/environments/Team01_Lab02_
extra-export reports/Team01_API_Automation_Report.html
↳ Initialize Payment Session
GET https://atqaa-medusa-store.onrender.com/store/carts/cart_01M709A0T3Y10H5ABCCNZC9S5W [200 OK, 3.24kB, 251ms]
'Automatically found regionId: reg_01M4BAM8S1CM467KDCWWDKVDWW'
GET https://atqaa-medusa-store.onrender.com/store/payment-providers?region_id=reg_01M4BAM8S1CM467KDCWWDKVDWW POST https://atqaa-medusa-store.onrender.com/store/payment-collections [20
0 OK, 578B, 147ms]
'Successfully set providerId: pp_system_default'
[200 OK, 605B, 479ms]
'Successfully set paymentCollectionId: pay_col_01M709A7VX32HEHVNC74M12JC3'
POST https://atqaa-medusa-store.onrender.com/store/payment-collections/pay_col_01M709A7VX32HEHVNC74M12JC3/payment-sessions [200 OK, 1.02kB, 551ms]
✓ [Tier 1] HTTP Status is successful (200 OK or 201 Created)
✓ [Tier 2] Response Time complies with SLA (< 2000ms)
✓ [Tier 3] Content-Type header is application/json
↳ Complete Order
POST https://atqaa-medusa-store.onrender.com/store/carts/cart_01M709A0T3Y10H5ABCCNZC9S5W/complete [200 OK, 8.82kB, 1286ms]
✓ [Tier 1] HTTP Status is successful (200 OK or 201 Created)
✓ [Tier 2] Response Time complies with SLA (< 2000ms)
✓ [Tier 3] Content-Type header is application/json
02_Admin_Authentication
↳ Admin Login Session
POST https://atqaa-medusa-store.onrender.com/auth/user/emailpass [200 OK, 924B, 779ms]
✓ [Tier 1] HTTP Status is successful (200 OK or 201 Created)
✓ [Tier 2] Response Time complies with SLA (< 2000ms)
✓ [Tier 3] Content-Type header is application/json
✓ Tier 2: token is a non-null string with JWT format
✓ Schema Valid
executed
failed
iterations 1 0
requests 15 0
test-scripts 19 0
prerequest-scripts 2 0
assertions 42 1
total run duration: 11.8s
total data received: 93kB (approx)
average response time: 771ms [min: 147ms, max: 2.1s, s.d.: 506ms]
failure detail
1. AssertionError [Tier 2] Response Time complies with SLA (< 2000ms)
expected 2112 to be below 2000
at assertion:1 in test-script
inside "01_Storefront_Catalog / Select Shipping Method"
khinthuzaraung@Khins-MacBook-Air ATQAA_Team01_Medusa_Automation %
Outline
Timeline
127.0.0.1:5500/reports/Team01_API_Automation_Report.html
Relaunch to update
Light
Dark
Summary
Total Requests 15
Failed Tests 1
Skipped Tests 0
Newman Run Dashboard
Monday, 09 November 2026 20:52:43
TOTAL ITERATIONS
1
TOTAL ASSERTIONS
42
TOTAL FAILED TESTS
1
TOTAL SKIPPED TESTS
0
FILE INFORMATION
Collection: Team01_Medusa_API_Testing
Environment: Medusa_Render_Staging_Env
TIMINGS AND DATA
Total run duration: 11.8s
Total data received: 90.82KB
Average response time: 771ms
SUMMARY ITEM
TOTAL
FAILED
Requests 15 0
Prerequest Scripts 2 0
Test Scripts 19 0
Assertions 42 1
Skipped Tests 0 -